

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Digital Signal and Image Processing (01CT1513)	Aim : Audio Convolution	
Open Ended Problem : 2	Date: 18/07/2026	Enrolment No: 92400133125

AIM : Audio Concolution

GitHub :

https://github.com/mankumarbhalodiya/Digital-Signal-and-Image-Processing/blob/main/Experiment%202/125_DSIP_OpenEnded_2.pdf

Code :

```
from pydub import AudioSegment
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve
# Load MP3 file
audio = AudioSegment.from_mp3(
"/content/Balam_Pichkari_Full_Song_Video_Yeh_Jawaani_Hai_Deewani__PRITAM__Ranbir_Kapoor,_
Deepika_Padukone(128k).mp3"
)
# Convert to mono and extract raw samples
audio = audio.set_channels(1)
samples = np.array(audio.get_array_of_samples()).astype(np.float32)
# Define convolution kernel
kernel = np.array([1, 0, 1, 0, 1], dtype=np.float32)
# Perform convolution
convoluted = convolve(samples, kernel, mode='same')
# Normalize to int16 range for saving
convoluted = convoluted / np.max(np.abs(convoluted)) # Normalize to [-1, 1]
convoluted_int16 = (convoluted * 32767).astype(np.int16) # Scale to int16
# Save as WAV using PyDub
convoluted_audio = AudioSegment(
convoluted_int16.tobytes(),
frame_rate=audio.frame_rate,
sample_width=2, # 16-bit samples = 2 bytes
channels=1
)
# Export to file
convoluted_audio.export("output_convoluted.wav", format="wav")
print("Convoluted audio saved as 'output_convoluted.wav'.")
# Plot original and convoluted signals (optional)
samples_norm = samples / np.max(np.abs(samples))
convoluted_norm = convoluted
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Digital Signal and Image Processing (01CT1513)	Aim : Audio Convolution	
Open Ended Problem : 2	Date: 18/07/2026	Enrolment No: 92400133125

```
plt.plot(samples_norm, color='blue')
plt.title("Original Audio Signal - Man Bhalodiya")
plt.subplot(2, 1, 2)
plt.plot(convoluted_norm, color='red')

plt.title("Convoluted Audio Signal with Kernel [1, 0, 1, 0, 1]")
plt.tight_layout()
plt.show()
```

Output :

